

readdir.C

```
1 // struct dirent - Structure that stores info about each file.
2 // readdir(dir) - read one file from directory
3 // d_name - store the file name
4
5
6 #include <stdio.h>
7 #include <dirent.h>
8
9 int main() {
10
11     DIR *dir;
12     struct dirent *entry;
13
14     dir = opendir(".");
15
16     if (dir == NULL) {
17         printf("Cannot open directory\n");
18         return 1;
19     }
20
21     printf("Files in directory:\n");
22
23     // Loop through all files in the directory
24     while ((entry = readdir(dir)) != NULL) { // readdir() reads one entry at a time
25         printf("%s\n", entry->d_name); // d_name stores the file name
26     }
27
28     closedir(dir);
29
30     return 0;
31 }
```

fork.C

```
1 // System call allows a program to request a service from the kernel
2
3 // Fork will create a new process (child) from an existing process (parent)
4 // After calling fork(): Two processes run the same code
5
6 // After fork() is executed:
7 // The child process becomes a copy of the parent process
8 // Both processes continue executing from the next line after fork()
9 // fork will return some special value if it is 0 then it is child, if > 0 then it is
  parent, if < 0 then fork is not executed
10
11 // unistd.h is a header file in C used mainly in UNIX/Linux systems.
12 // It provides declarations for many system calls and low-level operating system
  functions.
13 // unistd.h lets your C program communicate directly with the operating system.
14
15 #include <stdio.h>
16 #include <unistd.h>
17
18 int main() {
19
20     // child process execute the if block
21     if(fork() == 0) {
22         printf("This is CHILD process\n");
23     }
24
25     // parent process execute the else block
26     else {
27         printf("This is PARENT process\n");
28     }
29
30     printf("PID: %d\n", getpid());
31
32     return 0;
33 }
34
35
36 //      Parent Process
37 //      |
38 //      fork()
39 //      |   |
40 //      Parent Child
41
42
43
44
```

execl.C

```
1 // exec is a system call used to replace the current process with a new process.
2
3 // exec() → replaces the current process with a new process with same PID
4 // execl() → executes a file, with list of arguments
5
6
7 #include <stdio.h>
8 #include <unistd.h>
9
10 int main() {
11
12     printf("Before exec()\n");
13
14     execl("/bin/ls", "ls", NULL);
15
16     printf("After exec()\n");
17
18     return 0;
19 }
20
21
22 // "/bin/ls" - Path to the ls command on Linux
23 // "ls" - This becomes argv[0] in the new process
24 // NULL - End of Argument
25
26
```

readdir.C

```
1 // exec is a system call used to replace the current process with a new process.
2
3 // exec() → replaces the current process with a new process with same PID
4 // execl() → executes a file, with list of arguments
5
6
7 #include <stdio.h>
8 #include <unistd.h>
9
10 int main() {
11
12     printf("Before exec()\n");
13
14     execl("/bin/ls", "ls", NULL);
15
16     printf("After exec()\n");
17
18     return 0;
19 }
20
21
22 // "/bin/ls" - Path to the ls command on Linux
23 // "ls" - This becomes argv[0] in the new process
24 // NULL - End of Argument
25
26
```

pid.C

```
1 // A PID is a unique number given to every process by the operating system.
2 // Every process is created by another process (except the first system process).
3
4 // getpid() → gives program's process ID
5 // getppid() → gives PID of the process that started your program
6
7
8 #include <stdio.h>
9 #include <unistd.h>
10
11 int main() {
12
13     printf("PID: %d\n", getpid()); // PID of the current process
14     printf("PPID: %d\n", getppid()); // PID of the process that started your program
15
16     return 0;
17 }
18
19
20 // Parent could be:
21 // terminal, another program, shell, system service
22
23 // system
24 // |
25 // terminal
26 // |
27 // your program
```

opendir.C

```
1  #include <stdio.h>
2  #include <dirent.h> // for DIR, opendir, closedir
3
4  int main() {
5
6      DIR *dir;
7
8      dir = opendir("."); // opendir() opens a directory. "." means the current
                           directory
9
10     if (dir == NULL) { // if opendir() fails, it returns NULL
11         printf("Failed to open directory\n");
12         return 1;      // exit program with error code
13     }
14
15     printf("Directory opened successfully!\n"); // directory opened
16
17     closedir(dir);      // close the directory
18
19     return 0;
20 }
```